Universidad de Costa Rica

Escuela de Ciencias de la Computación e Informática

CI-0141 | G01

BASES DE DATOS AVANZADAS

Laboratorio 6

Bases de Datos NoSQL - Orientadas a Llave-Valor

Profesor:

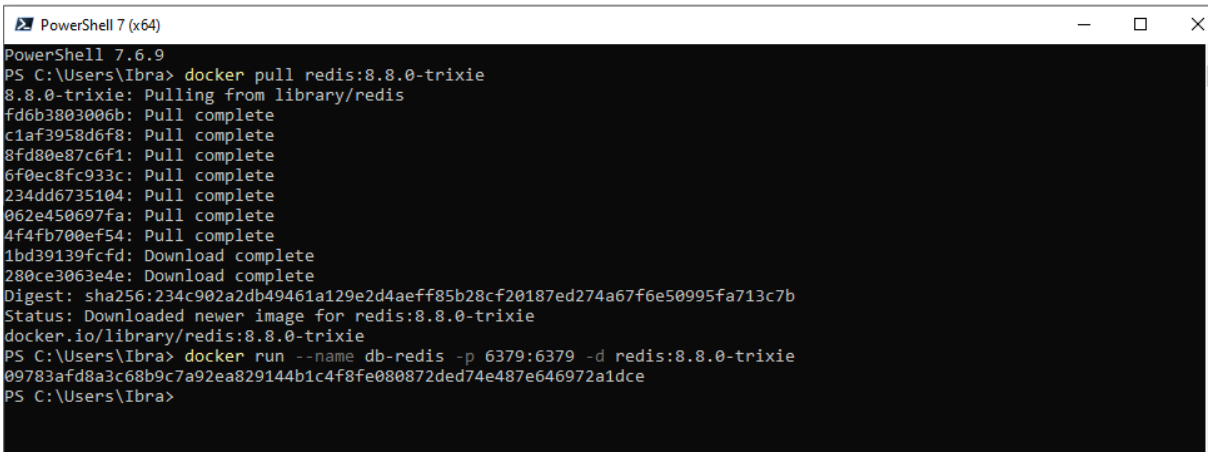
Mag. Luis Carlos Salas

Javier Cruz Ruiz C02517

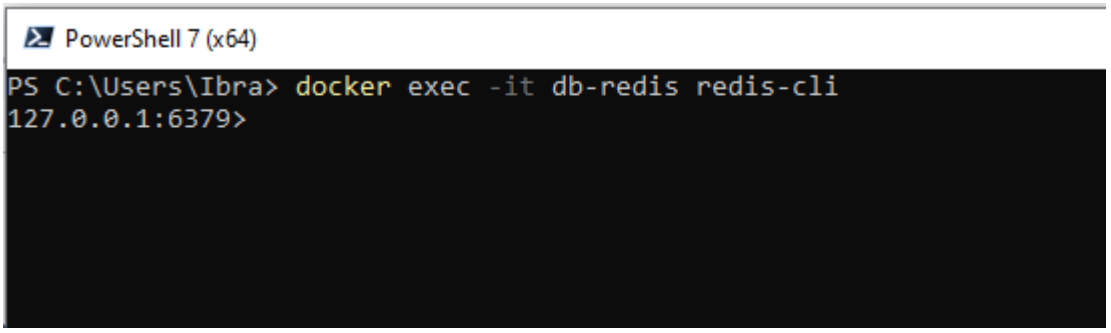
2 de Noviembre, 2026

Creación de instancia

1. Realice los siguientes pasos para realizar la habilitación de una instancia de Redis utilizando un contenedor en Docker.



2. Establezca una conexión la instancia de Redis utilizando la línea de comando.



Diseño de base de datos

1. Para cada uno de los grupos de datos anteriores, realice lo siguiente:

a. Proponga una convención para nombrar las llaves del grupo.

b. Escriba un ejemplo concreto de una llave que siga la convención propuesta. Si incluye identificadores variables, sustitúyalos por valores de ejemplo. Indique qué dato o colección de datos identificaría esa llave.

c. Proponga el tipo de valor que utilizaría para almacenar los datos del grupo.

d. Justifique brevemente la elección del tipo de valor, considerando las características de los datos y las operaciones que se realizarán sobre ellos.

Sesiones de usuario:

- **Convención para nombrar las llaves del grupo:** Se podría utilizar una convención del tipo sesion:<usuario>
- **Ejemplo:** Siguiendo la convención anterior, un ejemplo sería sesion:<jcruz>
- **Tipo de valor:** Podría utilizarse un **string** para almacenar los datos.
- **Justificación:** Cada usuario tiene un único identificador de sesión. Un String es suficiente y, al ser único por usuario, un nuevo SET sobre la misma llave reemplaza la sesión anterior.

Roles de usuario:

- **Convención para nombrar las llaves del grupo:** Se podría utilizar una convención del tipo roles:<usuario>
- **Ejemplo:** Siguiendo la convención anterior, un ejemplo sería roles:jcruz
- **Tipo de valor:** Podría utilizarse un **set** para almacenar los datos.
- **Justificación:** Set permite que un usuario pueda tener varios roles, pero sin repetir ninguno. Además, SMEMBERS consulta todos los roles, SISMEMBER verifica si tiene uno específico en tiempo constante, y SREM elimina un rol individual.

Espacios universitarios:

- **Convención para nombrar las llaves del grupo:** Se podría utilizar una convención del tipo espacio:<codigo>
- **Ejemplo:** Siguiendo la convención anterior, un ejemplo sería espacio:LAB-101
- **Tipo de valor:** Podría utilizarse un **Hash** para almacenar los datos.
- **Justificación:** Cada espacio es un objeto con varios atributos que se deben consultar y modificar individualmente. Un Hash permite leer todos los campos, solo algunos, actualizar uno, o eliminar uno, sin tocar el resto.

Solicitudes:

- **Convención para nombrar las llaves del grupo:** Se podría utilizar una convención del tipo solicitudes:<estado>
- **Ejemplo:** Siguiendo la convención anterior, un ejemplo sería solicitudes:pendientes
- **Tipo de valor:** Podría utilizarse un **List** para almacenar los datos.
- **Justificación:** Las solicitudes deben guardarse en una sola colección, List permite conservar el orden de llegada y atenderse de la más antigua a la más reciente.

2. Conteste la siguiente pregunta, ¿Por qué el diseño de una convención de nombres para llaves y la elección de los tipos de valores son partes importantes del modelado en una base de datos llave-valor?

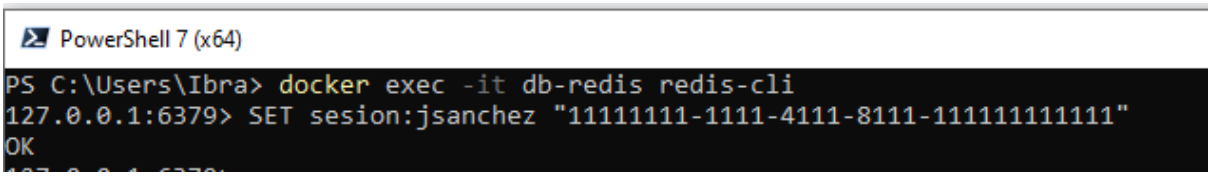
En una base llave-valor no hay tablas ni consultas como SQL, la única forma de encontrar un dato es conocer su llave. Por eso la convención de nombres funciona como el "esquema" de la base, ya que hace las llaves predecibles y legibles, evita colisiones entre grupos de datos y facilita su búsqueda y mantenimiento.

En este tipo de base de datos no se tienen tablas y consultas, como en SQL. La forma de encontrar un dato es conociendo su llave, es por esto que es importante tener una convención de nombres y conocer el esquema de la base de datos. Cada tipo de dato define qué operaciones permite realizar Redis sobre ese mismo dato.

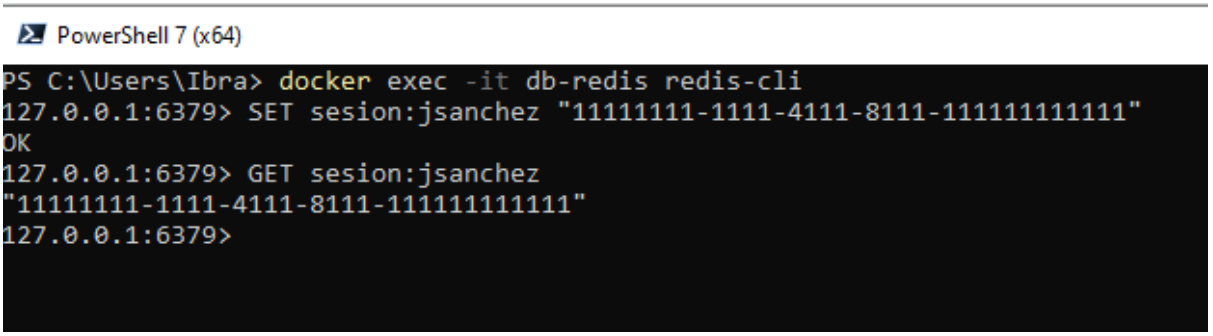
Pruebas de base de datos

1. Sesiones de usuario

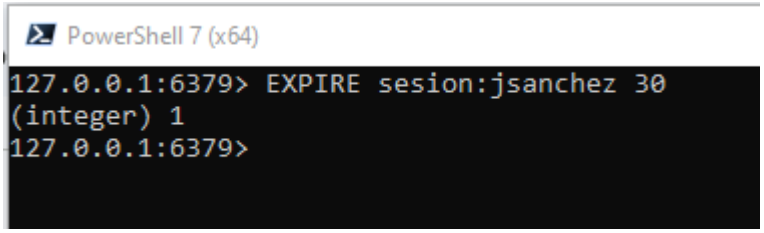
a. Cree una sesión para el usuario "jsanchez". El valor almacenado debe ser el siguiente identificador de sesión: "11111111-1111-4111-8111-111111111111".



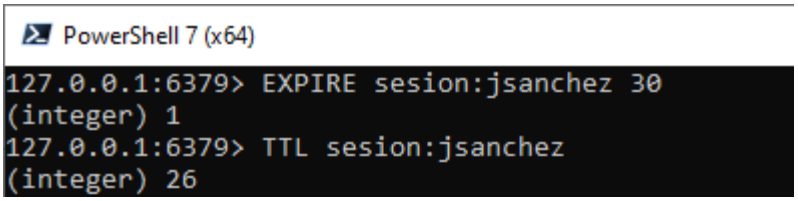
b. Consulte la sesión creada y verifique que el valor almacenado corresponde con el identificador de sesión.



c. Asigne un tiempo de vida limitado de 30 segundos a la sesión del usuario "jsanchez".



d. Consulte el tiempo de vida restante de la sesión.



e. Verifique si la llave de la sesión existe previo a su expiración.

PowerShell 7 (x64)

```
127.0.0.1:6379> EXPIRE sesion:jsanchez 30
(integer) 1
127.0.0.1:6379> TTL sesion:jsanchez
(integer) 26
127.0.0.1:6379> EXISTS sesion:jsanchez
(integer) 1
127.0.0.1:6379>
```

f. Espere a que finalice el tiempo de vida establecido para la sesión y verifique nuevamente si la llave de la sesión existe.

PowerShell 7 (x64)

```
127.0.0.1:6379> EXPIRE sesion:jsanchez 30
(integer) 1
127.0.0.1:6379> TTL sesion:jsanchez
(integer) 26
127.0.0.1:6379> EXISTS sesion:jsanchez
(integer) 1
127.0.0.1:6379> EXISTS sesion:jsanchez
(integer) 0
127.0.0.1:6379>
```

g. Cree una segunda sesión para el usuario “mrojas”, utilizando el siguiente identificador de sesión: “22222222-2222-4222-8222-222222222222”.

PowerShell 7 (x64)

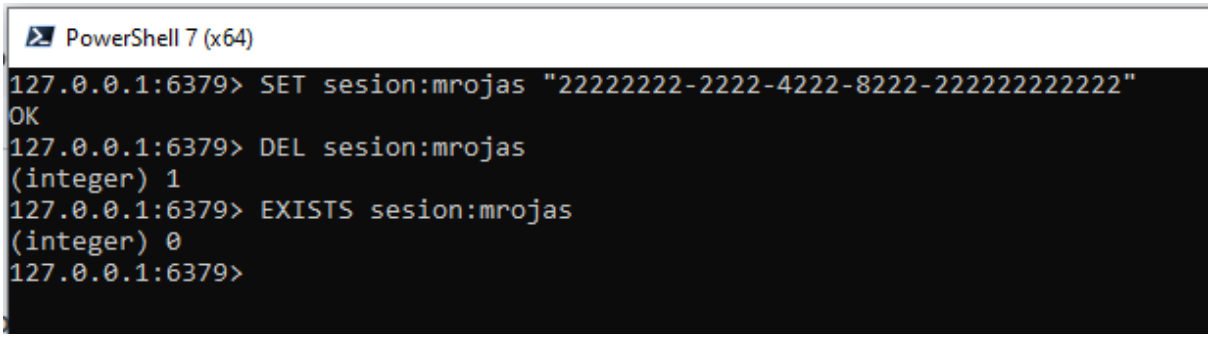
```
127.0.0.1:6379> SET sesion:mrojas "22222222-2222-4222-8222-222222222222"
OK
127.0.0.1:6379>
```

h. Elimine manualmente la sesión del usuario “mrojas”.

PowerShell 7 (x64)

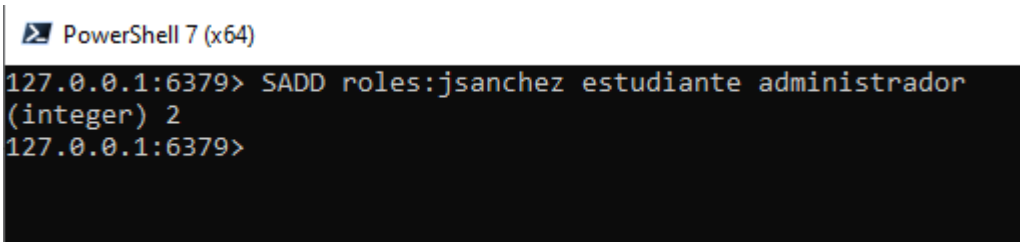
```
127.0.0.1:6379> SET sesion:mrojas "22222222-2222-4222-8222-222222222222"
OK
127.0.0.1:6379> DEL sesion:mrojas
(integer) 1
127.0.0.1:6379>
```

i. Verifique que la sesión del usuario “mrojas” ya no existe después de eliminarla.

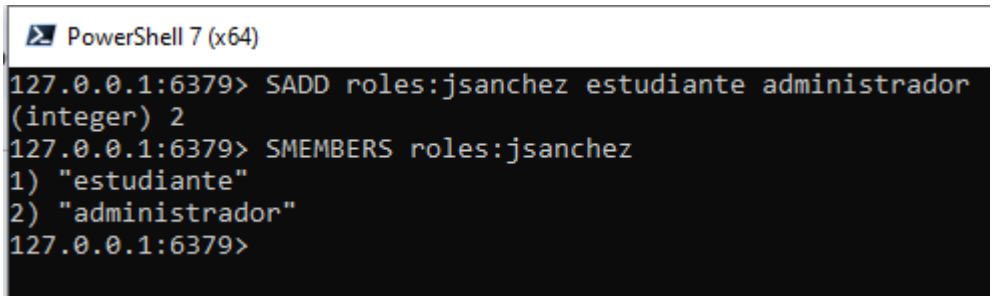


2. Roles de usuario

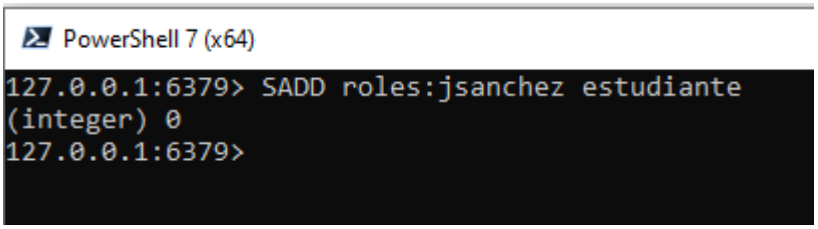
a. Registre para el usuario “jsanchez” los roles “estudiante” y “administrador”.



b. Consulte los roles registrados para el usuario “jsanchez”.



c. Intente registrar nuevamente el rol “estudiante” para el usuario “jsanchez”.



d. Consulte nuevamente los roles registrados para el usuario “jsanchez” y verifique si el rol repetido fue almacenado más de una vez.

PowerShell 7 (x64)

```
127.0.0.1:6379> SADD roles:jsanchez estudiante
(integer) 0
127.0.0.1:6379> SMEMBERS roles:jsanchez
1) "estudiante"
2) "administrador"
127.0.0.1:6379>
```

e. Verifique si el usuario “jsanchez” tiene asignado el rol “administrador”.

PowerShell 7 (x64)

```
127.0.0.1:6379> SISMEMBER roles:jsanchez administrador
(integer) 1
127.0.0.1:6379>
```

f. Verifique si el usuario “jsanchez” tiene asignado el rol “docente”.

PowerShell 7 (x64)

```
127.0.0.1:6379> SISMEMBER roles:jsanchez administrador
(integer) 1
127.0.0.1:6379> SISMEMBER roles:jsanchez docente
(integer) 0
127.0.0.1:6379>
```

g. Registre para el usuario “mrojas” los roles “docente” y “administrador”.

PowerShell 7 (x64)

```
127.0.0.1:6379> SADD roles:mrojas docente administrador
(integer) 2
127.0.0.1:6379>
```

h. Consulte los roles registrados para el usuario “mrojas”.

PowerShell 7 (x64)

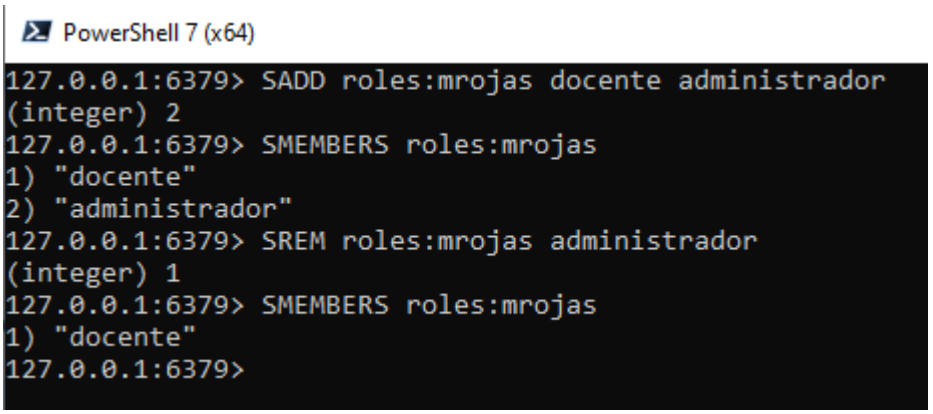
```
127.0.0.1:6379> SADD roles:mrojas docente administrador
(integer) 2
127.0.0.1:6379> SMEMBERS roles:mrojas
1) "docente"
2) "administrador"
127.0.0.1:6379>
```

i. Elimine el rol “administrador” del usuario “mrojas”.

PowerShell 7 (x64)

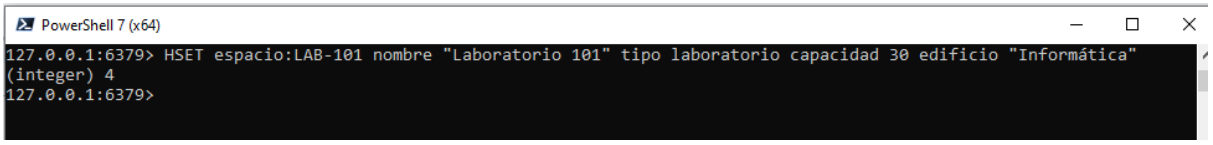
```
127.0.0.1:6379> SADD roles:mrojas docente administrador
(integer) 2
127.0.0.1:6379> SMEMBERS roles:mrojas
1) "docente"
2) "administrador"
127.0.0.1:6379> SREM roles:mrojas administrador
(integer) 1
127.0.0.1:6379>
```

j. Verifique nuevamente los roles registrados para el usuario “mrojas”.

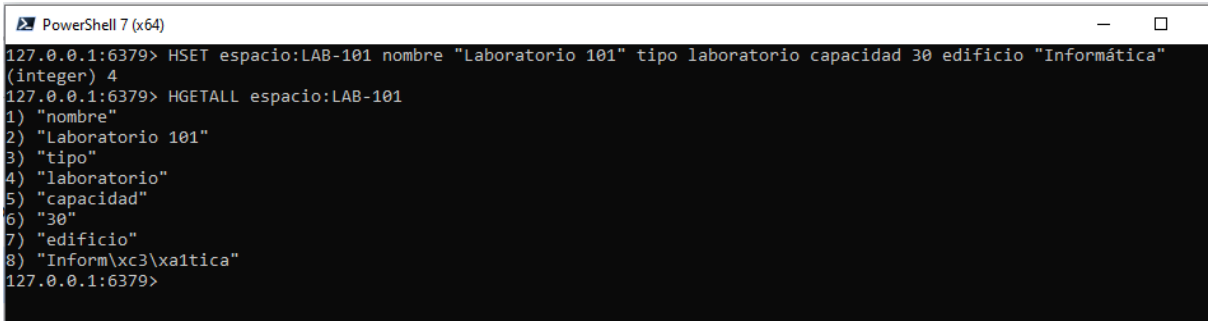


3. Espacios universitarios

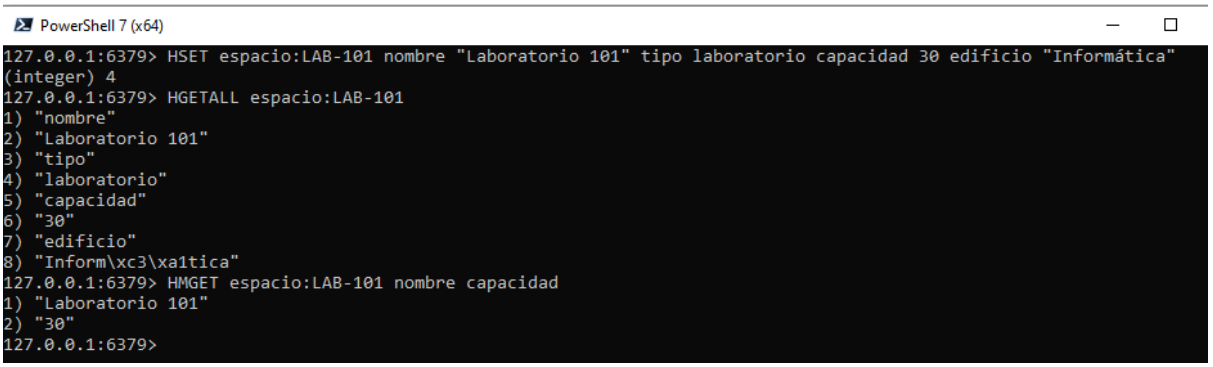
a. Registre un espacio universitario con código “LAB-101”. El espacio debe tener los siguientes datos: nombre “Laboratorio 101”, tipo “laboratorio”, capacidad “30” y edificio “Informática”.



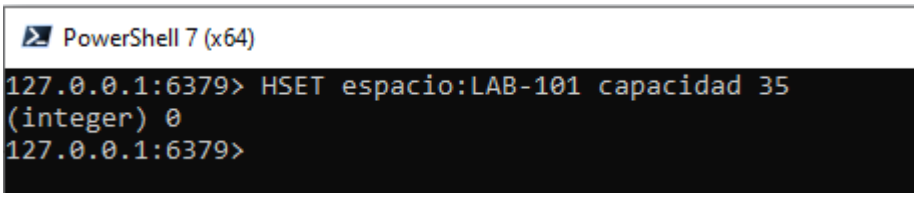
b. Consulte todos los datos registrados para el espacio “LAB-101”.



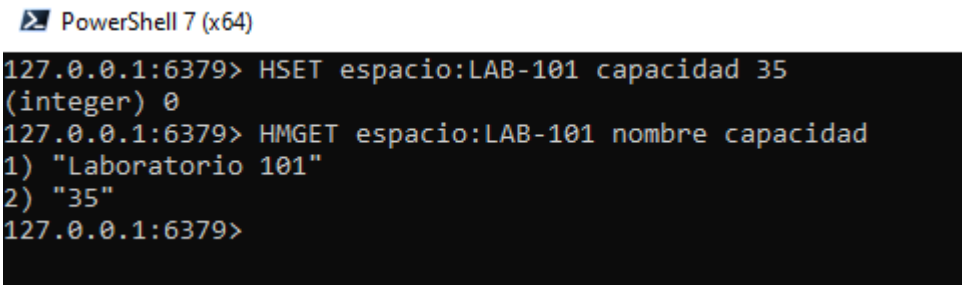
c. Consulte únicamente el nombre y la capacidad del espacio “LAB-101”.



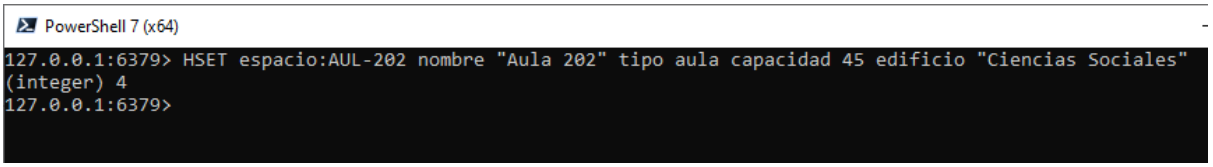
d. Actualice la capacidad del espacio “LAB-101” a “35”.



e. Consulte nuevamente el nombre y la capacidad del espacio “LAB-101” para verificar el cambio realizado.



f. Registre un segundo espacio universitario con código “AUL-202”. El espacio debe tener los siguientes datos: nombre “Aula 202”, tipo “aula”, capacidad “45” y edificio “Ciencias Sociales”.



g. Consulte todos los datos registrados para el espacio “AUL-202”.

```
PowerShell 7 (x64)
127.0.0.1:6379> HSET espacio:AUL-202 nombre "Aula 202" tipo aula capacidad 45 edificio "Ciencias Sociales"
(integer) 4
127.0.0.1:6379> HGETALL espacio:AUL-202
1) "nombre"
2) "Aula 202"
3) "tipo"
4) "aula"
5) "capacidad"
6) "45"
7) "edificio"
8) "Ciencias Sociales"
127.0.0.1:6379>
```

h. Elimine el campo “edificio” del espacio “AUL-202”.

```
PowerShell 7 (x64)
127.0.0.1:6379> HDEL espacio:AUL-202 edificio
(integer) 1
127.0.0.1:6379>
```

i. Consulte nuevamente todos los datos del espacio “AUL-202” para verificar que el campo fue eliminado.

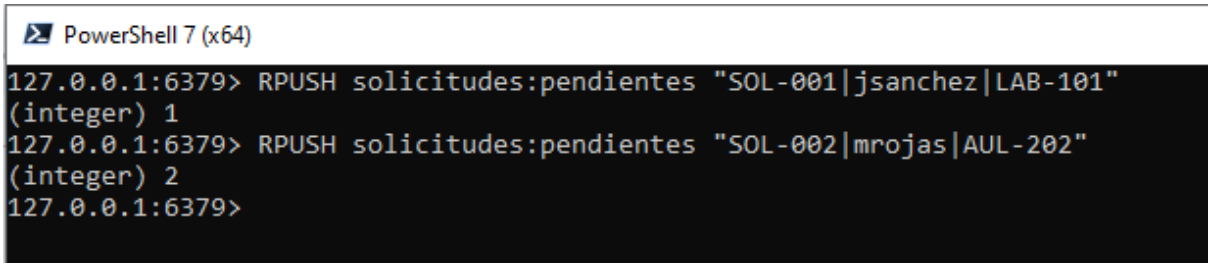
```
PowerShell 7 (x64)
127.0.0.1:6379> HDEL espacio:AUL-202 edificio
(integer) 1
127.0.0.1:6379> HGETALL espacio:AUL-202
1) "nombre"
2) "Aula 202"
3) "tipo"
4) "aula"
5) "capacidad"
6) "45"
127.0.0.1:6379>
```

4. Solicitudes

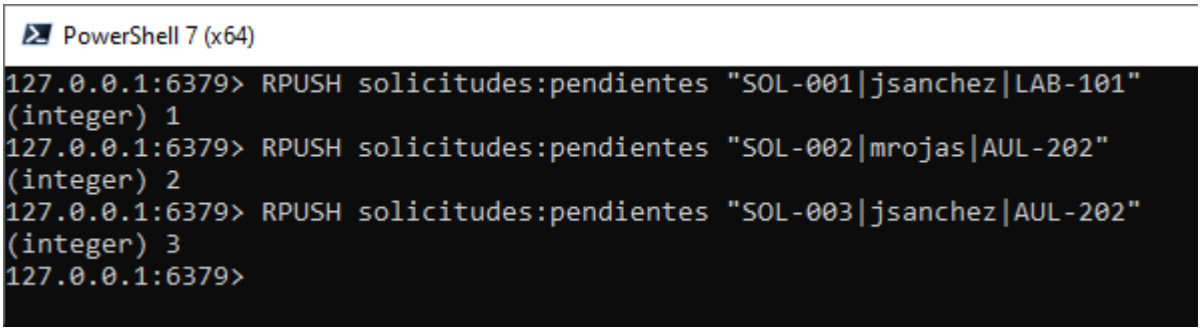
a. Registre una solicitud pendiente con el identificador “SOL-001”, realizada por el usuario “jsanchez” para solicitar el espacio “LAB-101”.

```
PowerShell 7 (x64)
127.0.0.1:6379> RPUSH solicitudes:pendientes "SOL-001|jsanchez|LAB-101"
(integer) 1
127.0.0.1:6379>
```

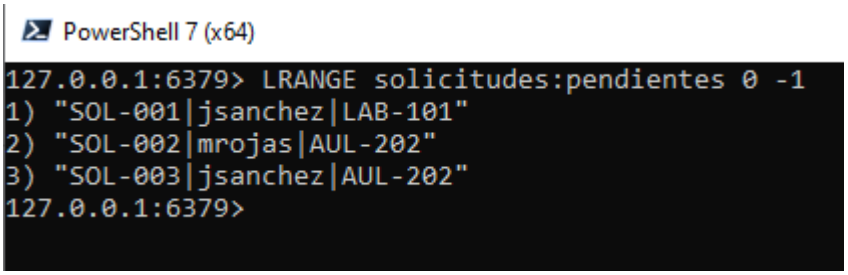
b. Registre una segunda solicitud pendiente con el identificador “SOL-002”, realizada por el usuario “mrojas” para solicitar el espacio “AUL-202”.



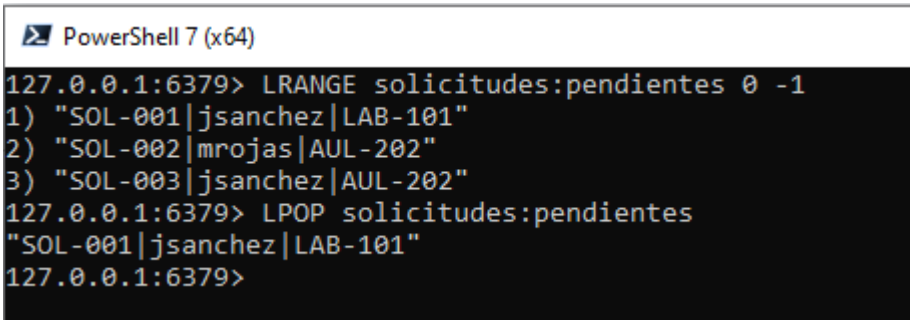
c. Registre una tercera solicitud pendiente con el identificador “SOL-003”, realizada por el usuario “jsanchez” para solicitar el espacio “AUL-202”.



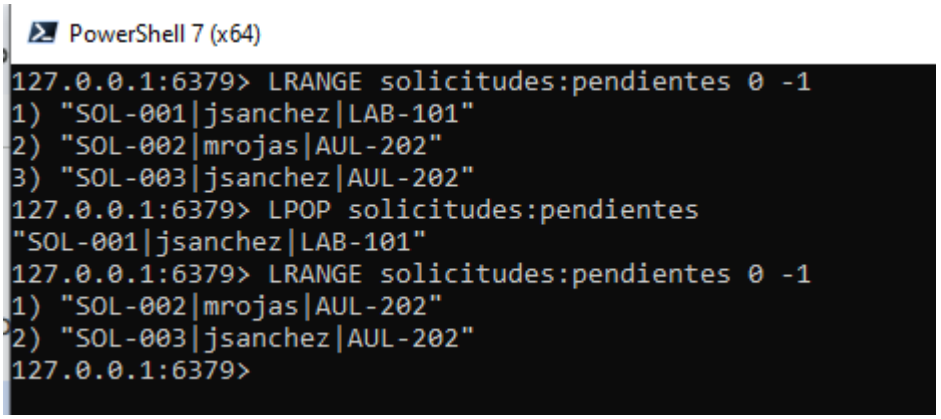
d. Consulte todas las solicitudes pendientes registradas.



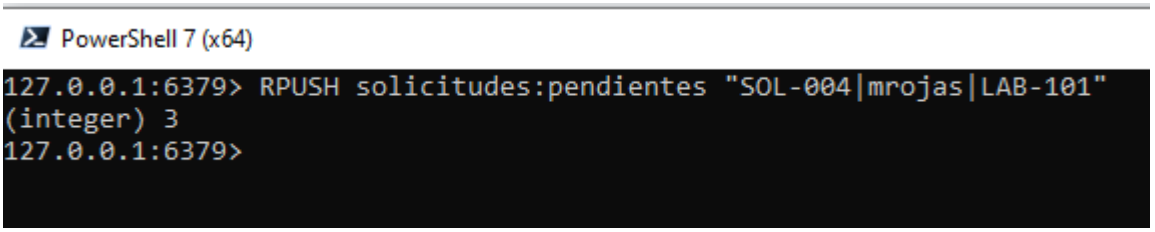
e. Procese la primera solicitud pendiente, considerando el orden en que las solicitudes fueron recibidas.



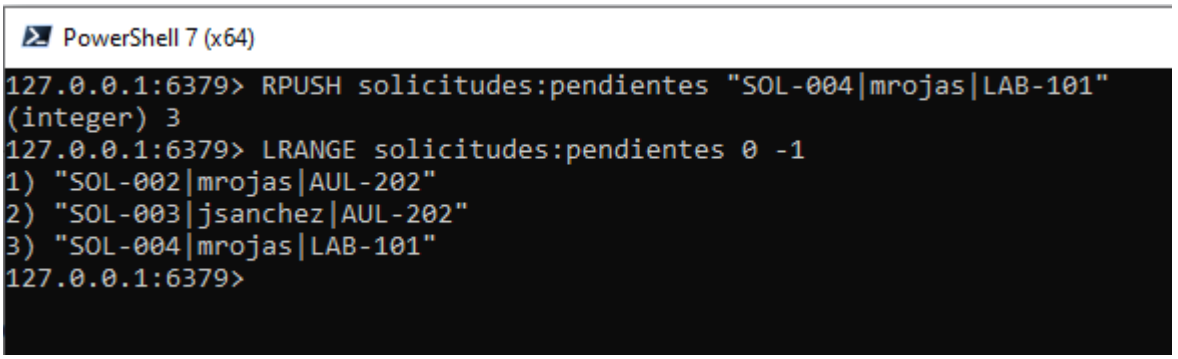
f. Consulte nuevamente las solicitudes pendientes para verificar que la solicitud procesada ya no se encuentra en la lista.



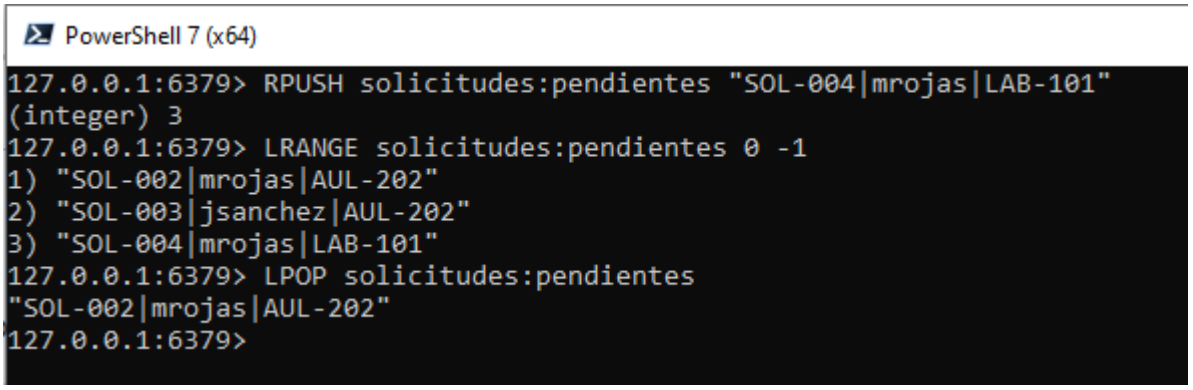
g. Registre una nueva solicitud pendiente con el identificador “SOL-004”, realizada por el usuario “mrojas” para solicitar el espacio “LAB-101”.



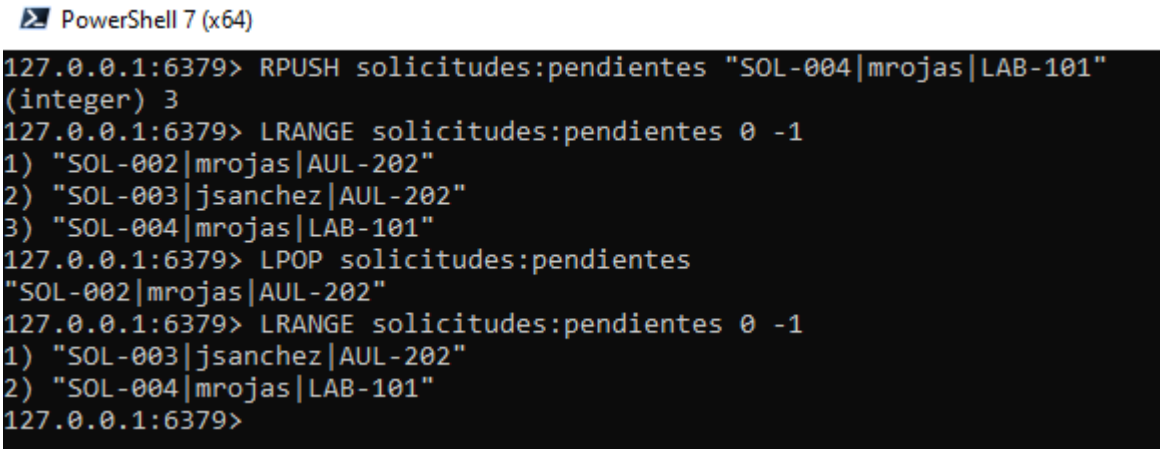
h. Consulte nuevamente todas las solicitudes pendientes registradas.



i. Procese otra solicitud pendiente, considerando nuevamente el orden en que las solicitudes fueron recibidas.



j. Consulte finalmente las solicitudes pendientes para verificar el estado de la lista.



Eliminación de instancia

1. Ejecute los siguientes comandos para eliminar el contenedor de la instancia de Redis. Asegúrese de haber terminado esta parte del laboratorio, ya que la eliminación no se puede deshacer.

